



Systèmes embarqués et robotique

Rapport de projet
Miniprojet sur le robot e-puck2

Encadré par:
Francesco Mondada, Daniel Burnier et Cyril Monette

Groupe G38
El-Hajj Olivier
Schwartz Maxime

Section: Microtechnique BA6

Printemps 2025

Table des matières

1	Introduction	3
2	Description détaillée du comportement du robot	3
2.1	FOLLOW_WALL	3
2.2	PARKING_FORWARD	4
2.3	TURNING_RIGHT	4
2.4	CHECKING_SPOT	4
2.5	SPOT_TAKEN	5
2.6	SPOT_FREE	5
2.7	PARKED	5
3	Utilisations techniques	5
3.1	Utilisation de la caméra	5
3.2	Capteurs infrarouges (IR)	6
3.3	Time Of Flight	6
4	Conclusion	6
5	Références	8

1 Introduction

Dans le cadre de notre cours de Robotique et Systèmes Embarqués [2], nous avons réalisé en binôme un miniprojet sur le robot e-puck2.

Pour ce projet, nous avons voulu réaliser un code qui soit en lien avec des besoins actuels : les voitures autonomes. Notre projet consiste à implémenter un code au robot qui lui permettrait de se garer automatiquement en effectuant les tâches suivantes :

- suivre un mur situé sur sa gauche
- détecter le début d'une place de parking par la présence d'un poteau (chaque place de parking est précédée d'un poteau)
- vérifier si cet emplacement est libre ou non
- si il n'est pas libre, aller jusqu'à la prochaine place
- effectuer une manoeuvre de stationnement dès qu'un emplacement libre est détecté

2 Description détaillée du comportement du robot

Voici une explication plus détaillée du fonctionnement de notre robot, et de la manière dont nous l'avons programmé.

Tout notre code est situé dans une boucle *while(1)*. Cette fonction représente une machine à états, qui va contrôler le comportement de notre robot de manière automatique. La boucle *while(1)* est une boucle qui va tourner de manière continue, afin de regarder à chaque tour dans quel *state* se trouve notre robot grâce à une structure *switch*.

Pourquoi une machine à états (FSM) ? Ce projet est naturellement séquentiel : notre robot doit effectuer des tâches dans un ordre bien défini (suivre un mur → détecter une place → se garer...). Une FSM permet d'organiser ces comportements de manière compréhensible, avec des transitions visibles. L'usage de threads à priorités différentes aurait inutilement complexifié la gestion des transitions des comportements.

Dans chaque state (excepté *PARKED*), une fonction particulière va être appelée qui a comme sortie le state pour la prochaine entrée dans la boucle *while(1)*. Ces fonctions sont toutes définies dans le fichier *FSM_states.c*.



FIGURE 1 – Schéma représentant la structure des states

2.1 FOLLOW_WALL

Au tout début, notre robot se situe dans la case *FOLLOW_WALL*. La fonction *follow_wall_state* est alors appelée. Dans cette fonction, l'état du booléen *is_poto_detected*, défini dans le fichier

poto_detection.c, est analysé. Ce fichier contient trois fonctions, dont deux qui permettent de changer l'état de ce booléen (*set_poto_detected* et *reset_poto_detected*) ainsi qu'une fonction qui permet d'accéder à ce booléen (*is_poto_detected*).

Tant que le booléen *is_poto_detected* est nul, cela signifie que le robot n'a pas détecté de poteau, ce qui implique le lancement de la fonction *align_with_wall* (définie dans le fichier *motors_control.c*). Cette fonction va utiliser en argument le *WALL_TARGET_DISTANCE*, une constante définie dans le fichier *ir_left.h* et qui représente la distance désirée avec le mur. La fonction *align_with_wall* utilise un régulateur PD, ce qui signifie qu'il va prendre en compte l'erreur ainsi que la dérivée de l'erreur dans ses équations, pour calculer un float correction, qui va être implémenté aux valeurs *BASE_SPEED* sur la roue droite et la roue gauche pour corriger cette erreur et rester aligné au mur. La fonction qui contrôle la vitesse des moteurs pas à pas est *set_motor_speeds* (définie dans le fichier *motors_control.c*). Le terme proportionnel K_p corrige l'écart instantané, alors que le terme dérivé K_d limite les oscillations et prévoit les variations rapides. Nous avons choisi un régulateur PD plutôt qu'un simple P, car le robot doit réagir rapidement à des écarts sans générer de très grandes corrections. Un PID complet n'était pas nécessaire puisqu'on ne s'intéresse pas à l'erreur statique. La valeur *BASE_SPEED* représente la vitesse de base du robot.

Lorsque le booléen *is_poto_detected* devient non nul, cela signifie qu'un poteau a été détecté, et le state passe alors à *PARKING_FORWARD*.

2.2 PARKING_FORWARD

La fonction *parking_forward_state* est appelée. Cette fonction fait avancer tout droit le robot, à une vitesse de *BASE_SPEED*, pendant une durée *PARK_FORWARD_DURATION* (définie dans le fichier *motors_control.h*). Cette durée représente le temps entre la détection d'un poteau, et le moment où il se tourne pour analyser l'emplacement. Lorsque ce temps est écoulé, le state passe alors à *TURNING_RIGHT*.

2.3 TURNING_RIGHT

La fonction *turning_right_state* est appelée. Cette fonction fait faire un tour de 90° au robot sur la droite. En effet, la roue gauche va tourner à une vitesse de *BASE_SPEED*, tandis que la roue droite va tourner à une vitesse de $-BASE_SPEED$, tout ça pendant une durée de *TURN_DURATION_MS*. Lorsque ce temps est écoulé, le robot va s'arrêter, utiliser la fonction *reset_poto_detection* pour pouvoir le réutiliser lors de la prochaine détection, et passer à la case *CHECKING_SPOT*.

2.4 CHECKING_SPOT

La fonction *checking_spot_state* est appelée. cette fonction renvoie à une autre fonction appelée *spot_is_taken_tof*, définie dans le fichier *TOF_car_detection*, qui renvoie un booléen en fonction de si l'emplacement est libre ou non. Pour cela, le Time Of Flight renvoie la distance libre en face du robot. Si cette valeur est inférieure à *TOF_OCCUPIED_THRESHOLD_MM*, alors l'emplacement n'est pas libre, le booléen renvoyé par la fonction sera non nul et le state passe alors à *SPOT_TAKEN*. En revanche, si la distance calculée par le Time Of Flight est supérieure à la valeur *TOF_OCCUPIED_THRESHOLD_MM*, l'emplacement est considéré

comme libre, le booléen renvoyé par la fonction sera nul, et le state passera alors à *SPOT_FREE*.

2.5 SPOT_TAKEN

La fonction *spot_taken_state* est appelée. Cette fonction va tout d'abord faire tourner le robot de 90° sur la gauche, afin de le réaligner avec le mur. Il réalise ce tour de la même manière que dans le state *TURNING_RIGHT*, la seule différence est que la roue gauche va tourner à une vitesse de $-BASE_SPEED$, tandis que la roue droite va tourner à une vitesse de $BASE_SPEED$. Ce tour dure également pendant *TURN_DURATION_MS*. Enfin, le state va alors repasser à *FOLLOW_WALL* dans le but de recommencer le processus.

2.6 SPOT_FREE

La fonction *spot_free_state* est appelée. Le but de cette fonction va être de permettre au robot de réaliser sa manœuvre de stationnement. La caméra va alors détecter une ligne verticale en face d'elle, et faire avancer le robot à une vitesse $BASE_SPEED$. Au moment où la largeur de la ligne verticale sera perçue du point de vue de la caméra comme supérieure à la valeur *LINE_WIDTH_THRESHOLD*, alors les moteurs s'arrêteront et le state passera à *PARKED*.

2.7 PARKED

Ce state ne fait rien de particulier, mais indique simplement que le robot a finalement réussi sa manoeuvre de stationnement.

3 Utilisations techniques

Une des contraintes principales de notre projet, citées dans la donnée [3], était l'utilisation de plusieurs éléments. Nous utilisons les moteurs pas à pas, deux capteurs de distance (IR et Time Of Flight) ainsi que la caméra. Vous trouverez dans cette section une description plus précise de la manière dont nous les avons utilisés.

3.1 Utilisation de la caméra

. Nous utilisons la caméra comme un capteur de distance. La fonction *extract_line_width*, définie dans le fichier *process_image.c*, introduite dans le TP4 [1]. Cette fonction va analyser l'intensité des pixels capturés par la caméra. Dès qu'une chute brutale de l'intensité des pixels est observée, cela indique qu'il y a eu un passage du clair au sombre, et donc le début de la ligne a été détecté. Puis, dès qu'un saut brutal de l'intensité des pixels est observé, cela signifie que la fin de la ligne a été détectée. Maintenant que l'on a la position initiale et finale de la ligne sombre, on peut alors calculer sa largeur en pixels. Grâce à une équation simple de trigonométrie (décrite en détail dans la solution du TP4 [1]), nous pouvons calculer la distance entre le robot et la caméra en fonction de la largeur de la ligne du point de vue du robot. Ainsi, notre robot continuera d'avancer tant que la largeur de la ligne renvoyée par la fonction *extract_line_width* ne dépasse pas la valeur *LINE_WIDTH_THRESHOLD*. Lorsque cette valeur est atteinte, le robot s'arrête et notre robot est considéré comme étant stationné.

La fonction *extract_line_width()* de *process_image.c* est issue de la librairie de nos travaux pratiques [1].

3.2 Capteurs infrarouges (IR)

Le robot e-puck2 est muni de huit capteurs infrarouges (IR1 (index 0) jusqu'à IR8 (index 7)). On utilise le capteur IR6 (gauche) pour l'alignement avec le mur et le capteur IR3 (droit) pour la détection de poteau.

1. Dans le fichier *ir_left.c*, on lit en continu la valeur mesurée par le capteur IR6 grâce au thread *IrLeftThread*. Ce dernier appelle la fonction *get_calibrated_prox()* de *proximity.c*, et stocke la valeur retournée dans une variable globale static accessible via un getter *get_left_distance()*. Puis la fonction *align_with_wall()* de *motors_control.c* utilise cette mesure pour appliquer une correction sur les vitesses des deux roues à l'aide d'un régulateur (PD), afin de garder une distance constante au mur.
2. Le capteur IR3 est lu par un thread *IrRightThread* défini dans *ir_right.c*. Dans ce thread, la valeur mesurée par le capteur est comparée en continu avec un certain seuil. Dès qu'elle le dépasse, le booléen partagé *poto_detected* est actualisé via *set_poto_detected()*, puis avec *reset_poto_detected()* quand le robot a passé le poteau. Nous avons ajouté une hystérésis afin d'éviter les fausses détections dues au bruit du capteur. Cette dernière permet de ne pas détecter plusieurs fois le même poteau quand la valeur mesurée par IR3 oscille autour du seuil, en imposant un écart suffisant avant de remettre le flag à zéro.

La fonction *get_calibrated_prox()* de *proximity.c* est issue de la librairie de nos travaux pratiques [1].

3.3 Time Of Flight

Le Time Of Flight (TOF, capteur VL53L0X) mesure le temps que met une onde pour partir du capteur, arriver à un objet et revenir au capteur, ce qui lui permet de déduire la distance entre le robot et l'objet en question. L'utilisation de ce capteur de distance est définie dans la fonction *spot_is_taken_tof*, située dans le fichier *TOF_car_detection*. On va alors comparer la valeur *TOF_OCCUPIED_THRESHOLD_MM* avec *dist_mm*, une autre valeur renvoyée par la fonction *VL53L0X_get_dist_mm*. C'est dans cette fonction que se situe le thread implémentant la valeur de *dist_mm*, grâce au Time Of Flight : à une fréquence de 100 ms, le thread va lire la dernière valeur du TOF et l'enregistrer dans *dist_mm*.

La fonction *VL53L0X_get_dist_mm()* de *VL53L0X.c* est issue de la librairie de nos travaux pratiques [1].

4 Conclusion

Difficultés rencontrées

- Le robot n'arrivait pas à détecter plus d'un poteau. Ce problème a été résolu lorsque l'on a rajouté la fonction *reset_poto_detected*.

- Le robot ne suivait pas correctement le mur avec un régulateur P. Ce problème a été résolu lorsque l'on a implémenté un régulateur PD.
- Ajustement de la ligne verticale nécessaire en face de la caméra, pour que le robot s'arrête à un endroit adéquat. Ce problème a été résolu grâce à une visualisation en direct via *plotImage.py*

Méthode de travail de groupe

Grace à une bonne communication, une répartition claire des tâches, une discussion à chaque self-assessment, des deadlines bien définies et une bonne utilisation de Git, nous avons pu nous répartir également le travail dans ce projet.

En conclusion, nous remercions les professeurs *Francesco Mondada*, *Daniel Burnier* et *Cyril Monette*, ainsi que l'ensemble de l'équipe d'assistants, pour leur accompagnement et leur soutien tout au long de ce projet.

5 Références

Références

- [1] MONETTE Cyril et BURNIER Daniel. Systèmes embarqués et robotique, répertoire GitHub des Travaux Pratiques. <https://github.com/EPFL-MICRO-315/TPs-Student>.
- [2] MONDADA Francesco. Cours de Systèmes embarqués et robotique. Lien moodle du cours : <https://moodle.epfl.ch/course/view.php?id=467>.
- [3] Francesco Mondada. Donnée miniprojet 2025. https://moodle.epfl.ch/pluginfile.php/22341/mod_resource/content/13/Donne%CC%81e%20miniprojet%202025.pdf, 2025.